Guide Git & Workflow Collaboratif – Cartino2D

Guide d'Intégration & Workflow Git – Équipe Cartino2D

Bienvenue sur le projet **Cartino2D (Cerema)** ! 🌊
Ce guide est destiné à **l'ensemble des collaborateurs et développeurs** de l'équipe (nouveaux arrivants comme utilisateurs réguliers).
Il explique comment récupérer le projet sur un nouveau poste, comment le configurer sans impacter les autres, et comment travailler à plusieurs sans écraser le travail de ses collègues.

📍 Où exécuter les commandes ?

Deux options sont possibles sur votre poste de travail :

Option A : Dans le Terminal intégré de RStudio (Recommandé)

1. Dans RStudio, regardez le panneau en bas à gauche.
2. Cliquez sur l'onglet **Terminal** (situé juste à côté de l'onglet *Console*).
3. Le terminal s'ouvre directement à la racine du projet (`C:\R\R-4.4.1\Cerema`). Tapez-y vos commandes `git ...` .

Option B : Via l'interface graphique de RStudio (Onglet Git)

1. Dans RStudio, regardez le panneau en haut à droite.
2. Cliquez sur l'onglet **Git**.
3. Vous y trouverez les cases à cocher (*Staged*), les boutons **Diff**, **Commit**, **Pull** (⬇️), **Push** (⬆️) et le menu déroulant des branches.

---

0. Prérequis indispensables (À faire une seule fois par poste)

Avant de cloner le dépôt, vérifiez que votre poste est prêt :

1. **Installer Git pour Windows** : Téléchargeable sur git-scm.com (laisser les options par défaut).
2. **Avoir un compte GitHub** et avoir été invité comme collaborateur sur le dépôt : https://github.com/CEREMA/cartino2d.
3. **Configurer votre identité Git** (dans PowerShell ou le Terminal de RStudio) :

```
git config --global user.name "Prénom Nom"
git config --global user.email "prenom.nom@cerema.fr"
```

(Sans cette configuration, Git bloquera vos commits ou les attribuera à un utilisateur anonyme).

---

1. Récupérer le code sur son poste (Onboarding d'un collègue)

Méthode 1 : Depuis RStudio (La plus simple et recommandée)

1. Ouvrez RStudio.
2. Allez dans le menu : **File → New Project... → Version Control → Git**.
3. Renseignez :
   - **Repository URL** : `https://github.com/CEREMA/cartino2d.git`
   - **Project directory name** : `Cerema` (ou le nom de votre choix)
   - **Create project as subdirectory of** : Choisissez votre dossier R (par ex. `C:\R\R-4.4.1`).
4. Cliquez sur **Create Project**.
5. RStudio télécharge tout le code et ouvre automatiquement le projet `Cartino2D.Rproj` avec l'onglet Git activé !

(Alternative en ligne de commande : `git clone https://github.com/CEREMA/cartino2d.git C:\R\R-4.4.1\Cerema` puis double-cliquez sur `Cartino2D.Rproj`).

⚠️ Étape obligatoire post-clone : Initialiser sa configuration locale

> ⚠️ **IMPORTANT : Pourquoi ces fichiers ne sont PAS sur GitHub ?**
> `C2D_LienOutilsPC.R` contient les chemins d'installation spécifiques à votre PC (ex: version de QGIS, de GRASS ou de Telemac).
> `C2D_CodeCalcuDistant.R` contient les identifiants et accès aux clusters de calcul.
> **Ils sont volontairement ignorés par Git** afin d'éviter qu'un collègue n'écrase par inadvertance les chemins ou mots de passe d'un autre !

Comment initialiser ces fichiers sur votre machine ?

1. Dans RStudio, ouvrez le script C2D_RUN.R et exécutez les premières lignes (ou lancez `source("C2D_RUN.R")`).
2. Le script va détecter l'absence de vos fichiers locaux et va **créer automatiquement** :
   - `C2D_ParamUser/C2D_LienOutilsPC.R` (copié depuis le modèle `C2D_LienOutilsPC_xxxxxx.R`).

- `C2D_ParamUser/C2D_CodeCalcuDistant.R` (copié depuis le modèle `C2D_CodeCalcuDistant_xxxxxx.R`).

3. **Ouvrez** `C2D_ParamUser/C2D_LienOutilsPC.R` et ajustez les chemins vers vos logiciels locaux (QGIS, GMSH, GRASS…).
4. *(Optionnel)* Si vous lancez des calculs distants sur cluster, renseignez vos identifiants dans `C2D_ParamUser/C2D_CodeCalcuDistant.R`.

Ces deux fichiers vous appartiennent : ils ne seront **jamais** envoyés sur GitHub grâce au `.gitignore`.

---

2. Créer sa branche de développement

Règle d'or de l'équipe : **on ne développe jamais directement sur la branche `main`**.
On travaille toujours sur sa propre branche afin de pouvoir tester et itérer sans risquer de casser le code des autres.

Convention de nommage des branches :

- Pour vos développements courants : `dev_<prenom>` ou `dev_<initiales>` (ex : `dev_nabil`, `dev_fp`, `dev_julien`, `dev_NH`).
- Pour une fonctionnalité ciblée : `feat_<nom_feature>` (ex : `feat_abaques_buse`, `feat_ign_lidar`).
- Pour corriger un bug précis : `fix_<nom_bug>` (ex : `fix_etape08_grass`).

En ligne de commande (Terminal RStudio) :

```
# 1. S'assurer de partir d'un 'main' propre et à jour
git checkout main
git pull origin main

# 2. Créer sa branche et basculer dessus
git checkout -b dev_monprenom

# 3. Publier sa branche sur GitHub (à faire une seule fois au début)
git push -u origin dev_monprenom
```

Via l'interface graphique RStudio :

1. Dans l'onglet **Git** (en haut à droite), cliquez sur l'icône **New Branch** *(deux carrés reliés)*.
2. Saisissez **Branch Name** : `dev_monprenom`
3. Cochez **Sync branch with remote**.
4. Cliquez sur **Create**.

---

3. Enregistrer et pousser ses modifications

Le cycle de validation Git comprend toujours 3 actions successives :
Staged (Préparer) → Commit (Enregistrer localement) → Push (Envoyer sur GitHub).

En ligne de commande (Terminal RStudio) :

```
# 1. Vérifier l'état de son espace de travail
git status

# 2. Préparer les fichiers modifiés à inclure
git add C2D/Cartino2D_Etape08_PostTelemacGrass.R
# (ou 'git add .' si vous avez vérifié avec 'git status'
        qu'aucun fichier indésirable n'est présent)

# 3. Valider le commit avec un message clair
git commit -m "feat(abaques): ajout de la prise en compte des
        buses obstruees"

# 4. Transmettre le commit sur votre branche GitHub
git push origin dev_monprenom
```

Via l'interface graphique RStudio :

1. Dans l'onglet **Git**, cochez la case **Staged** en face des fichiers modifiés que vous voulez enregistrer.
2. Cliquez sur le bouton **Commit** (coche verte).
3. Dans la boîte de dialogue, rédigez un message explicite dans la case **Commit message** (en haut à droite).
4. Cliquez sur le bouton **Commit** en bas à droite.
5. Cliquez ensuite sur la flèche verte ⬆️ **Push** en haut à droite pour envoyer vos modifications sur GitHub.

---

4. Fusionner sa branche dans `main` (Mise en commun)

Pour intégrer vos développements dans la branche principale officielle, deux méthodes existent.

Méthode A : Via une Pull Request (Recommandée pour l'équipe)

La **Pull Request (PR)** est la bonne pratique collaborative car elle permet aux collègues de relire le code, d'échanger et de valider ensemble avant d'intégrer les changements dans `main`.

1. Rendez-vous sur la page GitHub du projet : https://github.com/CEREMA/cartino2d.
2. GitHub affiche un bandeau jaune indiquant que vous venez de pousser sur votre branche : cliquez sur **« Compare & pull request »**.
3. Renseignez un titre et décrivez brièvement les modifications apportées.

4. Cliquez sur le bouton vert **« Create pull request »**.
5. Après relecture éventuelle avec un collègue, cliquez sur **« Merge pull request »** puis **« Confirm merge »**.
6. **Mettre à jour votre poste local ensuite :** Dans le Terminal RStudio :

```
git checkout main
git pull origin main
```

Méthode B : Directement en local (Si vous validez vous-même)

Si vos modifications sont urgentes et validées, vous pouvez faire la fusion directement depuis le Terminal RStudio :

```
# 1. Basculer sur main
git checkout main

# 2. Récupérer les éventuelles nouveautés distantes
git pull origin main

# 3. Fusionner votre branche dans main
git merge dev_monprenom

# 4. Pousser le main mis à jour sur GitHub
git push origin main

# 5. Revenir sur votre branche de travail
git checkout dev_monprenom
```

(Ou directement depuis la Console R avec l'invite `>` *) :*

```
system("git checkout main ; git pull origin main ; git merge
        dev_monprenom ; git push origin main ; git checkout
        dev_monprenom")
```

---

5. Routine quotidienne & Bonnes pratiques d'équipe

🌞 La routine du matin (Avant de commencer à coder)

Avant de lancer une journée de travail, il est indispensable de récupérer le travail éventuel fait par vos collègues la veille, et de l'intégrer dans votre branche de travail :

```
# 1. Se mettre sur main et récupérer les dernières nouveautés de
        l'équipe
git checkout main
git pull origin main

# 2. Revenir sur sa branche
git checkout dev_monprenom

# 3. Fusionner main dans sa branche
git merge main
```

> 💡 **ASTUCE :** En faisant ce réflexe `git merge main` régulièrement dans votre branche, vous travaillez toujours sur un code à jour et vous éliminez 99% des risques de conflits lors de la fusion finale !

🛡️ Fichiers protégés & `.gitignore`

Le `.gitignore` officiel de Cartino2D est configuré pour protéger le dépôt : - **Ne JAMAIS forcer l'ajout (`git add -f`)** de fichiers de configuration machine (`C2D_LienOutilsPC.R`, `C2D_CodeCalcuDistant.R`). - **Ne JAMAIS commiter de gros fichiers de simulation** : `.res`, `.slf`, `.asc`, `.msh`, `.laz`, etc. - Les fichiers de secteurs créés pour vos études locales (ex: `C2D_ParamUtilisateur_MonSecteur.R`) sont automatiquement ignorés par Git pour ne pas polluer le dépôt.

✍️ Messages de commit : les conventions de l'équipe

Privilégiez des petits commits fréquents et ciblés avec des préfixes normalisés : - `feat( ... )` : Nouvelle fonctionnalité (ex: `feat(prec2d): ajout telechargement dalle lidar ign`) - `fix( ... )` : Correction d'un bug (ex: `fix(etape08): raccourcissement chemin grassdata pour Windows`) - `docs( ... )` : Documentation ou commentaires (ex: `docs: mise a jour du guide install`) - `refactor( ... )` : Restructuration ou optimisation sans changement fonctionnel

---

⚔️ Gestion d'un conflit de fusion (Conflict)

Si Git vous indique un conflit lors d'un `merge` ou d'un `pull` : 1. Tapez `git status` pour repérer le fichier concerné. 2. Ouvrez le fichier dans RStudio : Git affiche clairement les deux versions : `text <<<<<<< HEAD Votre version du code ======= La version venant de main ou d'un collègue >>>>>>> main` 3. Discutez avec le collègue si besoin, conservez le bon code, supprimez les balises `<<<<<<<`, `=======` et `>>>>>>>`. 4. Enregistrez et validez : `powershell git add nom_du_fichier.R git commit -m "fix: resolution conflit de fusion sur nom_du_fichier.R"`

📑 Mémo des commandes indispensables

Besoin	Commande Terminal RStudio
Voir l'état de mon dossier	`git status`
Voir les différences de code	`git diff`
Changer de branche	`git checkout nom_branche`
Créer une nouvelle branche	`git checkout -b nom_branche`
Mettre à jour main depuis GitHub	`git pull origin main`
Intégrer main dans ma branche	`git merge main`
Préparer un fichier modifié	`git add C2D/mon_script.R`
Enregistrer localement (Commit)	`git commit -m "feat: description"`
Envoyer sur GitHub (Push)	`git push origin nom_branche`
Voir l'historique des 5 derniers commits	`git log --oneline -n 5`